TypeScript Interview Questions and Answers

Last Updated : 23 Jul, 2025

TypeScript, a robust, statically typed **superset of JavaScript**, has become a go-to language for building scalable and maintainable applications. Developed by Microsoft, it enhances JavaScript by adding static typing and modern ECMAScript features, enabling developers to catch errors early and improve code quality. Leading companies such as Evernote, LinkedIn, Microsoft, and Meta leverage TypeScript for its reliability, scalability, and performance.

Here, we present **50+ essential TypeScript interview questions and answers**, tailored for both **freshers** and **experienced professionals** with **3, 5, or 10 years of experience**. Topics range from object-oriented programming and type inference to error handling and decorators, ensuring you're well-prepared to excel in your TypeScript interviews in 2025.

Table of Content

***Note**: Before diving into TypeScript interview questions and answers, if you're new to the language, we recommend building a solid foundation first by exploring our free TypeScript Tutorial.*

TypeScript Interview Questions for Freshers

1. Explain the data types available in TypeScript.

There are mainly two types of data types available in TypeScript:

1. **Built-in data types:** These data types already exist in typescript. They can be directly used to define the variables with different values.

- **String:** It represents a text value like "GeeksforGeeks", or "Computer Science".
- **Number:** It represents the numbered values i.e. 2, 28, 99, etc.
- **Boolean:** It stores **true** or **false** values.
- **Null:** An empty value deliberately assigned to a variable.
- **Undefined:** Represents a variable that is declared but not initialized.
- **any:** Represents any value of any data type and any number of values of different data types.
- **void:** Used to represent that a particular function is not going to return any value of any data type.

2. **User-defined data types:** These are the data types that are defined by the user they may contain multiple values of multiple data types.

- **arrays:** In typescript, arrays are used to store the multiple values of any kind of data type.
- **enums:** A special class that specifies the constant variables.
- **classes:** Used to store different data type values in the form of key-value pairs.
- **Interface:** These represent the basic syntax and blueprint that an entity must adhere to.

2. In how many ways we can declare variables in TypeScript?

In TypeScript, there are three ways in which we can declare variables:

- **Using the var keyword:** It is the oldest way of declaring varibles.

```
var name: string = "GeeksforGeeks";
```

- **Using the let keyword:** It was introduced in ES6 or ECMAScript 2015. It is a secure way of declaring variables than using **var,** as it has block scope.

```
let name: string = "GeeksforGeeks";
```

- **Using the const keyword:** It was also introduced in ES6. It is used to declare the constant variable whose values will not change throughout the code execution.

```
const name: string = "GeeksforGeeks"
```

2. In how many ways we can declare variables in TypeScript?

In TypeScript, there are three ways in which we can declare variables:

- **Using the var keyword:** It is the oldest way of declaring varibles.

```
var name: string = "GeeksforGeeks";
```

- **Using the let keyword:** It was introduced in ES6 or ECMAScript 2015. It is a secure way of declaring variables than using **var,** as it has block scope.

```
let name: string = "GeeksforGeeks";
```

- **Using the const keyword:** It was also introduced in ES6. It is used to declare the constant variable whose values will not change throughout the code execution.

```
const name: string = "GeeksforGeeks"
```

3. How you can declare a explicit variables in Typescript?

In typeScript, you can declare static variables using the colons **(:)** followed by the data type of the explicit type. You can not assign a value of some other data type to a static variable. The values of the same data type can be assigned.

Syntax:

```
let variable_name: data-type = value;
```

Example:

```
let company_name: string = "GeeksforGeeks";
company_name = "Cricket";
console.log(company_name); // Prints Cricket

company_name = 28;
console.log(company_name);
// Throws an error: Type '28' is not assignable to type 'string'.
```

4. How to declare a function with typed annotation in TypeScript?

In TypeScript, you can declare the functions by defining the type of parameters the take and the type of parameter it will return after the execution.

Example:

```
function annotedFunc(myName: string, age: number): string{
    return `My name is ${myName} and my age is ${age}.`;
}
console.log(annotedFunc("Emrit", 22));

// Prints: My name is Emrit and my age is 22.
console.log(annotedFunc("Neha", "18"));

// Above statement throws an error:
// Argument of type '"18"' is not assignable to parameter of type 'number'.
```

5. Describe the "any" type in TypeScript.

The **any** data type will allow you to assign the value of any data type to a variable. Sometimes, when the data is coming from other resouces like API call or user entered data. In that case, you may not aware of the type of the data so you can use the any data type to assign the value of any kind to a variable.

Example:

```
let studentData: string = `{
    "studentName": "Aakash",
    "studentID": 12345,
    "studentCourse": "B. Tech"
}`;
let student: any = JSON.parse(studentData);
```

5. Describe the "any" type in TypeScript.

The **any** data type will allow you to assign the value of any data type to a variable. Sometimes, when the data is coming from other resouces like API call or user entered data. In that case, you may not aware of the type of the data so you can use the any data type to assign the value of any kind to a variable.

Example:

```
let studentData: string = `{
    "studentName": "Aakash",
    "studentID": 12345,
    "studentCourse": "B. Tech"
}`;
let student: any = JSON.parse(studentData);

console.log(student.studentName, student.studentID, student.studentCourse);
// Prints: Aakash 12345 B. Tech
```

6. What are the advantages of using TypeScript?

There are a lot of advantages of using TypeScript, some of them are listed below:

- The TypeScript code can compile down to the JavaScript code that is runnable on every browser.
- It allow us to declare strongly or statically typed variables.
- It consist of advanced features like code completion, intelliSense etc.
- It supports namespace concept with the help of modules.
- TypeScript throw errors at the compilation time during development unlike of JavaScript that shows errors at the runtime.

7. List some disadvantages of using TypeScript.

There also exist some disadvantages of using TypeScript as listed below:

- The concept of abstract classes is not supported by TypeScript.
- Code Compilation is a time taking process in TypeScript.
- A extra step of converting the TypeScript code into JavaScript code requires while running TypeScript.
- A definition file needs to be added for using any external or third party library. All the external libraries not have the definition file.
- The quality of all the definition files need to be correct.

8. Explain the void type in TypeScript.

It is just opposite of **any** type. The **void** type represents the unavailability of the data type for any variable. It is mainly used with the functions that returns nothing. The variables defined using the void keyword can only be assigned with the **null** and **undefined** values.

Example:

```
function favGame(): void{
    console.log("My Favourite game is Cricket.");
}
favGame();
// Prints: My Favourite game is Cricket.
```

9. What is type null and its use in TypeScript?

The **null** keyword is considered as a data type in TypeScript as well as in JavaScript. The null keyword basically indicates the unavailability of a value. It can be used to check whether a value is provided to a particular variable or not.

Example:

```
function getData(orgName: string | null, orgDesc: string | null): void {
    if (orgName === null || orgDesc === null) {
        console.log("Not enough values provided to print.");
    }
    else {
        console.log(`Organization Name: ${orgName},
        \nOrganization Description: ${orgDesc}`);
    }
}
```

9. What is type null and its use in TypeScript?

The **null** keyword is considered as a data type in TypeScript as well as in JavaScript. The null keyword basically indicates the unavailability of a value. It can be used to check whether a value is provided to a particular variable or not.

Example:

```
function getData(orgName: string | null, orgDesc: string | null): void {
    if (orgName === null || orgDesc === null) {
        console.log("Not enough values provided to print.");
    }
    else {
        console.log(`Organization Name: ${orgName},
        \nOrganization Description: ${orgDesc}`);
    }
}

getData(null, null);
getData("GeeksforGeeks",
    "A Computer Science Portal.");
```

Output:

```
Not enough values provided to print.
Organization Name: GeeksforGeeks,
Organization Description: A Computer Science Portal.
```

10. Describe the syntax for creating objects in TypeScript.

A object is basically a collection of key-value pairs, where each key needs to be unique. In TypeScript, the objects can be created by declaring the property name and the type it is going to store.

Example:

```
const myObj: { name: string, desc: string } = {
    name: "GeeksforGeeks",
    desc: "A Computer Science Portal"
};
console.log(myObj);
// Prints: { name: 'GeeksforGeeks', desc: 'A Computer Science Portal' }
```

11. Can we specify the optional properties to TypeScript Object, if Yes, explain How?

Yes, we can declare the TypeScript Objects by specifying the optional properties that may or may not be defined inside the object. We can declare these peoperties by using a **?** symbol just after the property name while creeating the object.

Example:

```
const myObj: { name: string, desc: string, est?: number } = {
    name: "GeeksforGeeks",
    desc: "A Computer Science Portal",
};
console.log(myObj);
// Prints: { name: 'GeeksforGeeks', desc: 'A Computer Science Portal' }
myObj.est = 2008;
console.log(myObj);
// Prints: { name: 'GeeksforGeeks', desc: 'A Computer Science Portal' , est:
2008}
```

12. Explain the undefined type in TypeScript.

The concept **undefined** in TypeScript is similar to the concept of undefined in JavaScript. The undefined is used to indicate a variable which is declared but not assigned a value and it's either hoisted or in temporal dead zone. The memory to these kind of variables is allocated in the memory allocation phase and the **undefined** value is given to them until a value is assigned by the developer or programmer.

```
2008}
```

12. Explain the undefined type in TypeScript.

The concept **undefined** in TypeScript is similar to the concept of undefined in JavaScript. The undefined is used to indicate a variable which is declared but not assigned a value and it's either hoisted or in temporal dead zone. The memory to these kind of variables is allocated in the memory allocation phase and a **undefined** value assigned to them untill a value is assigned by the developer or programmer.

13. Explain the behavior of arrays in TypeScript.

The arrays defined in typescript are different from JavaScript and they also behave differently from JavaScript arrays. In typescript, the arrays are defined by specifying the static data types and can only store the multiple values of a single kind of data type.

Example:

```
const typedArray1:number[] = [1, 23, 28, 56];console.log(typedArray1);
  // 1, 23, 28, 56
const typedArray2:number[] = [1, 23, 28, 56, "GeeksforGeeks"];
console.log(typedArray2);
 // Throws an error: Type 'string' is not assignable to type 'number'.
```

14. How can you compile a TypeScript file?

To compile a TypeScript file, you use the tsc (TypeScript Compiler) command. You can compile a TypeScript file by running:

```
tsc filename.ts
```

This command compiles the .ts file into a .js (JavaScript) file. Make sure to have TypeScript installed globally or locally in your project for this to work.

15. Differentiate between the .ts and .tsx file extensions given to the TyppeScript file.

The **.ts** file extension is used to create a file that contains the pure TypeScript code inside it. These files are mainly created to implement the classes, functions, reducers and other pure typescript code. These files does not contain any JSX code. On the other hand, the **.tsx** file extensions are used to create the files that contains the JSX code inside it. These files are mainly used to build a react component that returns the JSX code at the end.

16. What is "in" operator and why it is used in TypeScript?

The **in** operator is used to check whether a property is present in the testing object or not. It will return true, if it finds the property in the object. Otherwise, it will return false.

17. Explain the union types in TypeScript?

The union types in TypeScript represents that the value of a variable can be one of the specified types. The union type uses a **straight vertical bar(|)** to show the options for the variable types.

18. Explain type alias in TypeScript?

A type alias in TypeScript is used to give a new and the meaning fule name for a combined or new type. The type alias will not create a new type instead it create new names for the type it refers to.

```
type combinedType = number | boolean
```

19. Is TypeScript strictly statically typed language?

No, TypeScript is not a strictly statically typed language it is an optional statically typed language that means it is in our hands that a particular variable has to be statically typed or not. We can use the **any** type and allow a variable to accept the value of any kind of data type. We can also define a variabe with a particular data type that a variable can accept and throws error if a value of some other data type is assigned to it.

20. Is template literal supported by TypeScript?

Yes, template literal is supported by TypeScript. We can interpolate a string using the template literals syntax (``) in TypeScript. The values of different variables can be shown inside a string using ${variable_name} syntax while interpolating with the template literals.

other data type is assigned to it.

20. Is template literal supported by TypeScript?

Yes, template literal is supported by TypeScript. We can interpolate a string using the template literals syntax (``) in TypeScript. The values of different variables can be shown inside a string using **${variable_name}** syntax while interpolating strig using template literals.

21. How to declare a arrow function in TypeScript?

In TypeScript, we can declare the arrow functions in the same format as we declare in vanilla JavaScript. TypeScript allow us to use the typed declaration with by specifying the type of parameters and the type of return value as well.

Example:

```
const typedArrowFunc = (org_name: string, desc: string): string => {
    let company: string = `Organization: ${org_name}, Description: ${desc}`;
    return company;
}
console.log(typedArrowFunc("GeeksforGeeks", "A Computer Science Portal"));
// Prints: Organization: GeeksforGeeks, Description: A Computer Science Portal
```

22. How to define a function which accepts the optional parameters?

You can define a function with optinal parameters by using a **?** symbol in the declaration of the function for the parameter which you want to make the optional as shown below:

Example:

```
function cricketer(c_name: string, runs?: number): void{
    if(!runs){
        console.log(`Cricketer Name: ${c_name}, Runs Scored: Not Available`);
    }
    else{
        console.log(`Cricketer Name: ${c_name}, Runs Scored: ${runs}`);
    }
}
cricketer("Virat Kohli", 26000);
cricketer("Yuzvender Chahal");
// Prints:
// Cricketer Name: Virat Kohli, Runs Scored: 26000
// Cricketer Name: Yuzvender Chahal, Runs Scored: Not Available
```

23. Explain noImplicitAny in TypeScript.

The **noImplicitAny** is a compiler option that we can add to the tsconfig.json file and make the TypeScript compiler to throw an error if a function or method for a particular type is invoked on a parameter of any type.

Generally, TypeScript expect a explicit type to be associated with the parameters, but somethimes we don't need to specify the explicit type. In that case, TypeScript assigns a explicit type as **any type** to that parameter and allows to perform operations. But we can force the compiler to throw an error if a explicit type is not defined for a parameter.

24. What are interfaces in TypeScript?

A interface in TypeScript is used to define a syntax that must be followed by the entity of that interface. An Interface defines the properties, methods and the events and considered as the members of the interface. The interfaces only contain the declarations of the members. The initialisation or the assignment will be done by the class that is deriving the interface. Interfaces are defined using the **interface** keyword.

25. In how many ways you can use the for loop in TypeScript?

There are mainly three ways in which you can use the for loop in TypeScript as listed below:

- **Using the simple for loop:** It is the simple for loop used to iterate through any number of variables by defining and using a variable.

```
for(let i=0; i<n; i++){ // code statement}
```

- **Using the for-of loop:** It can be used to iterate through the array elements without defining the iterator variable.

defined using the **interface** keyword.

25. In how many ways you can use the for loop in TypeScript?

There are mainly three ways in which you can use the for loop in TypeScript as listed below:

- **Using the simple for loop:** It is the simple for loop used to iterate through any number of variables by defining and using a variable.

```
for(let i=0; i<n; i++){ // code statement}
```

- **Using the for-of loop:** It can be used to iterate through the array elements without defining the iterator variable.

```
for(let item of myArr){ // code statement}
```

- **Using the forEach() method:** It is a method that takes a callback function and operate the functionality to each item of the array.

```
myArr.forEach(() => { // code statement})
```

TypeScript Intermediate Interview Questions

26. What is never type and its uses in TypeScript?

A **never** type in typescript is indicate the values that may never be occured. It is mainly used with the function that return nothing and always thrown an exception or error. A **never** type is different from **void** type. Because, a function that returns nothing implicitly returns **undefined** and these functions are inferred using the **void** keyword. But a function that declared using the **never** keyword will never return a undefined it only returns never type. The never type can be used with following cases:

- With an infinite loop.
- In a function that throws exceptions or errors.

Example:

```
function neverFunc(): never{
    // Function Statements
}
```

27. Explain the working of enums in TypeScript?

An **enum** in typescript is used to create a collection of constants. It is basically a class that allow us to create multiple constants of **numeric** as well as **string type.** By default, the value of numeric constant starts from **0** and increases accordingly for every constant by a margin of **1**. You can also change the initialisation value from 0 to any other value of your choice. It is declared using the **enum** keyword followed by the name of enum and constants.

Example:

```
enum demoEnum{
    milk = 1,
    curd,
    butter,
    cheese
}
```

```
}
```

27. Explain the working of enums in TypeScript?

An **enum** in typescript is used to create a collection of constants. It is basically a class that allow us to create multiple constants of **numeric** as well as **string type.** By default, the value of numeric constant starts from **0** and increases accordingly for every constant by a margin of **1**. You can also change the initialisation value from 0 to any other value of your choice. It is declared using the **enum** keyword followed by the name of enum and constants.

Example:

```
enum demoEnum{
    milk = 1,
    curd,
    butter,
    cheese
}
let btr: demoEnum = demoEnum.butter;
console.log(btr)
// Prints: 3
```

28. Explain the parameter destructuring in TypeScript.

The parameter destructuring is nothing more than the unpacking of the provided or passed object properties individually into the one or more parameters when the object is passed to an function. It can be done as shown below:

```
function getOrganisation({ org_name, org_desc }: { org_name: string, org_desc:
string }) {
    console.log(`Organization Name: ${org_name}, \nOrganization Description:
${org_desc}`);
}
getOrganisation({ org_name: "GeeksforGeeks", org_desc: "A Computer Science
Portal." });

// Prints:
// Organization Name: GeeksforGeeks,
// Organization Description: A Computer Science Portal.
```

Example:

```
interface interface_name{
    // Define the members like methods properties and events etc.
}
```

29. Explain type inference in TypeScript.

The type inference refers to the automatically assigning the explicit types to the variable which does not declared using the expicit type. Generally, it is done at the time when the variables are declared and initialized at the same time with a value of some data type.

30. What are modules in TypeScript?

Modules are used to create a collection of multiple data types that may include the classes, functions, interfaces and variables. The modules have their own scope. The members defined inside modules can not be accessed directly by the other code. Modules are imported using the **import** statement and defined using the **export** keyword to export it.

Example:

```
module multiply{
    export function product(a: number, b: number): number{
        return a*b
    }
}
```

31. In how many ways you can classify Modules?

There are two types of Modules available in TypeScript:

- **Internal Modules:** These modules are used to specify the collection of classes, interfaces, functions and variables that can be exported t ther modules as a single unit.
- External Modules: These are the separate files that include more code and consist of

```
}
```

31. In how many ways you can classify Modules?

There are two types of Modules available in TypeScript:

- **Internal Modules:** These modules are used to specify the collection of classes, interfaces, functions and variables that can be exported to other modules as a single unit.
- **External Modules:** These are the separate TypeScript files that include more code and consist of at least one export or import statement in it.

32. What is the use of tsconfig.json file in TypeScript?

This file helps to compile the project by providing the compiler options. It also makes the working directory as the root directory of the project.

33. What are Decorators in TypeScript?

In TypeScript, decorator is a syntax that can be used to define the function, classes, properties etc. to modify their behaviour and add some metadata to them. These are basically the higher order function that takes target element as argument and returns the modified version of it with the help of some function.

34. How to debug a TypeScript file?

A TypeScript file can be debugged using the **--sourcemap** command that is followed by the file name. It will create a new file with the **fileName.ts.map** name.

```
tsc --sourcemap script.ts
```

35. Describe anonymous functions and their uses in TypeScript?

The anonymous functions are the functions that are declared without a name. These functions are mainly used to pass to another function as a call back function like while attaching an event and calling the setTimeout() method etc.

Example:

```
myBtn.addEventListener('click', function () {
    // Add click functionality
});
```

Example:

```
let unionVar: number | boolean = true;
console.log(unionVar); // true

unionVar = 56;
console.log(unionVar); // 56
```

36. Is it possible to call the constructor function of the base class using the child class?

Yes, the base class constructor can be called using the **super()** method inside the constructor function of the child class with the required parmeters if the base class constructor function takes parameters.

37. How to combine multiple TypeScript files and convert them into single JavaScript file?

There is a command named **--outfile** that is follwed by the JavaScript filename and the multiple TypeScript files. If the JavaScript file name is not provided then all the TypeScript files are combined in the first TypeScript file specified in the format.

```
tsc --outfile combined.js script1.ts script2.ts script3.ts
```

38. Explain type of operator in TypeScript and where to use it.

The **typeof** operator is used to check or get the type of a particular variable. It can also be used to set the similar explicit type to another variables.

Example:

```
tsc --outfile combined.js script1.ts script2.ts script3.ts
```

38. Explain type of operator in TypeScript and where to use it.

The **typeof** operator is used to check or get the type of a particular variable. It can also be used to set the similar explicit type to another variables.

Example:

```
const strVar: string = "GeeksforGeeks";
const numVar: number = 28;
console.log(typeof strVar, typeof numVar); //Prints: string number

const numVar2: typeof numVar = 25;
const strVar2: typeof strVar = "Cricket";
console.log(typeof strVar2, typeof numVar2); //Prints: string number
```

39. How you can compile a TypeScript file?

The TypeScript code is not executed directly. It requires to convert the TypeScript code into JavaScript. You can use **tsc <file_name>** command to execute the TypeScript code and convert it into JavaScript.

```
tsc script.ts
```

40. Which principles of Object Oriented Programming are supported by TypeScript?

TypeScript supports all the four principles of Object Oriented Programming as:

- Abstraction
- Encapsulation
- Inheritance
- Polymorphism

41. Explain Mixins in TypeScript

The mixins are used to create the classes from the reuseable components. They can be built using the different partial classes. In simple language, we can say that instead of a class extends another class, a function can take a class and then return the class as a result. The function that takes the class and returns a class is known as mixin.

42. Is it possible to create the immutable Object properties in TypeScript?

Yes, you can use the **readonly** property before the properties names while defining the objects. It makes the properties of object to be initialized at the time of declaring the object. If you try to update the value of any immutable property it will not allow you to do it as it is a readonly property.

TypeScript Interview Questions For Experienced

43. In what situation you should use a class and a interface?

We can use the classes and interfaces to define our own custom data types. There are different use cases and situations where we can use the interfaces and classes.

An interface can be used in a situation where the shape, structure and the contact will be defined for indicating how a object or class will look like without their implementation. It can also be used to enforce some properties and methods to a class and object.

On the other hand, a class can be used where we need to enclose the data to hide it from outer code and prevent the direct acces of this data. It can be used to implement the Object Oriented Programmin concepts.

44. What are the differences between the classes and the interfaces in TypeScript?

A class is defined using the **class** keyword. The classes can contain the methods, properties and variables. Methods of a class are defined when the class is implemented. A class instance will allow us to access the properties and methods defined inside the class.

Interfaces are defined using the interface keyword. It contains only the declarations of the properties and methods which are implemented by the derived class.

45. How to declare a class in TypeScript?

The syntax for declaring the classes in TypeScript is the same as in JavaScript. In TypeScript, you can also use the typed declarations for the variables and methods of the class.

properties and methods which are implemented by the derived class.

45. How to declare a class in TypeScript?

The syntax for declaring the classes in TypeScript is almost same as in JavaScript. In TypeScript, you can also use the typed declarations for the variables and methods of the class.

Example:

```
 class Cricketer{
    name: string;
    runs: number;

    constructor(name: string, runs: number){
        this.name = name;
        this.runs = runs;
    }

    thisMatchRund(): number{
        this.runs += 139;
        return this.runs;
    }
}
```

46. How the inheritance can be used in TypeScript?

Inheritance is one of the main four pillars of Object Oriented Programming. It allows a class to inherit or acquire the properties and methods of other class and implement them with the instance created using the inherited class. The inheritance can be implemented using the **extends** keyword after the child class name followed by the parent class name.

```
class Child extends Parent{
    // Properties of child class
}
```

47. What are the different ways for controlling the visibility of member data?

TypeScript provide us with the three ways to control the visibility of members like methods and properties in classes.

- **Private:** The private members can be accessed only inside the class. They can not be accessed by the outer code.
- **Public:** It is the default visibilty of the members. These members can be access from any where in the code by defining the class instance.
- **Protected:** These members can only be accessed by the classes that are inherited using the members class. No other code that does not inherit the class can access these members.

48. How to convert a .ts file into TypeScript Definition file?

You can change a .ts file into the definition file with the help of TypeScript compiler by using the **--declaration** command followed by the name od the .ts file. The definition file will make the TypeScript file reuseable.

```
tsc --declaration script.ts
```

49. Is it possible to create the static classes in TypeScript?

No, TypeScript does not supports the static classes. Because, in typescript does not require the static classes as you can simply create any function or data as a object without creating a containing class.

50. Expalin conditional typing in TypeScript?

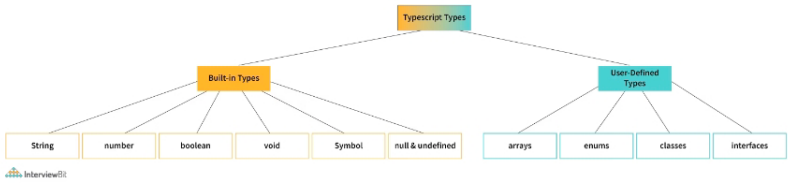
In TypeScript, we can use the ternary operator(condition? true:false) to assign the dynamic types to an property. It will assign a type dynamically based on the condition defined in the ternary operator.

Typescript Interview Questions for Freshers

1. What are the primitive types in TypeScript?

TypeScript has three primitive types that are frequently used: string, number, and boolean. These correspond to the similarly named types in JavaScript.

- **string**: represents text values such as “javascript”, “typescript”, etc.
- **number**: represents numeric values like 1, 2, 32, 43, etc.
- **boolean:** represents a variable that can have either a ‘true’ or ‘false’ value.



2. Explain how the arrays work in TypeScript.

We use arrays to store values of the same type. Arrays are ordered and indexed collections of values. The indexing starts at 0, i.e., the first element has index 0, the second has index 1, and so on.

Here is the syntax to declare and initialize an array in TypeScript.

```
let values: number[] = [];
values[0] = 10;
values[1] = 20;
values[2] = 30;
```

You can also create an array using the short-hand syntax as follows:

```
let values: number[] = [15, 20, 25, 30];
```

TypeScript provides an alternate syntax to specify the Array type.

```
let values: Array<number> = [15, 20, 25, 30];
```

3. What is any type, and when to use it?

There are times when you want to store a value in a variable but don’t know the type of that variable in advance. For example, the value is coming from an API call or the user input. The ‘any’ type allows you to assign a value of any type to the variable of type any.

```
let person: any = "Foo";
```

Here is an example that demonstrates the usage of any type.

```
// json may come from a third-party API
const employeeData: string = `{"name": "John Doe", "salary": 60000}`;

// parse JSON to build employee object
const employee: any = JSON.parse(employeeData);

console.log(employee.name);
console.log(employee.salary);
```

TypeScript assumes a variable is of type **any** when you don’t explicitly provide the type, and the compiler cannot infer the type from the surrounding context.

4. What is void, and when to use the void type?

The void indicates the absence of type on a variable. It acts as the opposite type to any. It is especially useful in functions that don’t return a value.

```
function notify(): void {
  alert("The user has been notified.");
```

the surrounding context.

4. What is void, and when to use the void type?

The void indicates the absence of type on a variable. It acts as the opposite type to any. It is especially useful in functions that don't return a value.

```
function notify(): void {
  alert("The user has been notified.");
}
```

If a variable is of type void, you can only assign the null or undefined values to that variable.

5. What is an unknown type, and when to use it in TypeScript?

The unknown type is the type-safe counterpart of any type. You can assign anything to the unknown, but the unknown isn't assignable to anything but itself and any, without performing a type assertion of a control-flow-based narrowing. You cannot perform any operations on a variable of an unknown type without first asserting or narrowing it to a more specific type.

Consider the following example. We create the foo variable of unknown type and assign a string value to it. If we try to assign that unknown variable to a string variable bar, the compiler gives an error.

```
let foo: unknown = "Akshay";
let bar: string = foo; // Type 'unknown' is not assignable to type 'string'.(2322)
```

You can narrow down a variable of an unknown type to something specific by doing typeof checks or comparison checks or using type guards. For example, we can get rid of the above error by

```
let foo: unknown = "Akshay";
let bar: string = foo as string;
```

6. What are the different keywords to declare variables in TypeScript?

var: Declares a function-scoped or global variable. You can optionally set its value during the declaration. Its behavior and scoping rules are similar to the var keyword in JavaScript. For example,

```
var foo = "bar";
```

let: Declares a block-scoped local variable. Similar to var, you can optionally set the value of a variable during the declaration. For example,

```
let a = 5;

if (true) {
  let a = 10;
  console.log(a);  // 10
}
console.log(a);  // 5
```

const: Declares a block-scoped constant value that cannot be changed after it's initialized. For example,

```
const a = 5;

if (true) {
  a = 10; // Error: Cannot assign to 'a' because it is a constant.(2588)
}
```

7. Explain the arrow function syntax in TypeScript.

Arrow functions provide a short and convenient syntax to declare functions. They are also called lambdas in other programming languages.

Consider a regular function that adds two numbers and returns a number.

```
function add(x: number, y: number): number {
let sum = x + y;
return sum;
}
```

Using arrow functions syntax, the same function can be defined as:

```
let add = (x: number, y: number): number => {
let sum = x + y;
return sum;
}
```

You can further simplify the syntax by getting rid of the brackets and the return statement. This is allowed when the function body consists of only one statement. For example, if we remove the temporary sum variable, we can rewrite the above function as:

```
let add = (x: number, y: number): number => x + y;
```

Arrow functions are often used to create anonymous callback functions in TypeScript. Consider the example below that loops over and filters an array of numbers and returns an array containing multiples of five. The filter function takes an arrow function.

```
let numbers = [3, 5, 9, 15, 34, 35];

let fiveMultiples = numbers.filter(num => (num % 5) == 0);

console.log(fiveMultiples);  // [5, 15, 35]
```

Arrow functions are often used to create anonymous callback functions in TypeScript. Consider the example below that loops over and filters an array of numbers and returns an array containing multiples of five. The filter function takes an arrow function.

```
let numbers = [3, 5, 9, 15, 34, 35];

let fiveMultiples = numbers.filter(num => (num % 5) == 0);

console.log(fiveMultiples);  // [5, 15, 35]
```

8. Provide the syntax of a function with the type annotations.

Functions are blocks of code to perform a specific code. Functions can optionally take one or more arguments, process them, and optionally return a value.

Here's the TypeScript syntax to create and call a function.

```
function greet(name: string): string {
  return `Hello, ${name}`;
}

let greeting = greet("Anders");
console.log(greeting);  // "Hello, Anders"
```

9. How to create objects in TypeScript?

Objects are dictionary-like collections of keys and values. The keys have to be unique. They are similar to arrays and are also sometimes called associative arrays. However, an array uses numbers to index the values, whereas an object allows you to use any other type as the key.

In TypeScript, an Object type refers to any value with properties. It can be defined by simply listing the properties and their types. For example,

```
let pt: { x: number; y: number } = {
  x: 10,
  y: 20
};
```

10. How to specify optional properties in TypeScript?

An object type can have zero or more optional properties by adding a '?' after the property name.

```
let pt: { x: number; y: number; z?: number } = {
  x: 10,
  y: 20
};
console.log(pt);
```

In the example above, because the property 'z' is marked as optional, the compiler won't complain if we don't provide it during the initialization.

11. Explain the concept of null and its use in TypeScript.

In programming, a null value indicates an absence of value. A null variable doesn't point to any object. Hence you cannot access any properties on the variable or call a method on it.

In TypeScript, the null value is indicated by the 'null' keyword. You can check if a value is null as follows:

```
function greet(name: string | null) {
if (name === null) {
  console.log("Name is not provided");
} else {
  console.log("Good morning, " + name.toUpperCase());
}
}

var foo = null;
greet(foo); // "Name is not provided"

foo = "Anders";
greet(foo);  // "Good morning, ANDERS"
```

12. What is undefined in TypeScript?

When a variable is declared without initialization, it's assigned the undefined value. It's not very useful on its own. A variable is undefined if it's declared, but no value has been assigned to it. In contrast, null is assigned to a variable, and it represents no value.

```
console.log(null == null); // true
console.log(undefined == undefined); // true
console.log(null == undefined); // true, with type-conversion
console.log(null === undefined); // false, without type-conversion
console.log(0 == undefined); // false
console.log('' == undefined); // false
console.log(false == undefined); // false
```

13. Explain the purpose of the never type in TypeScript.

As the name suggests, the never type represents the type of values that ne

```
console.log('' == undefined); // false
console.log(false == undefined); // false
```

13. Explain the purpose of the never type in TypeScript.

As the name suggests, the never type represents the type of values that never occur. For example, a function that never returns a value or that always throws an exception can mark its return type as never.

```
function error(message: string): never {
throw new Error(message);
}
```

You might wonder why we need a 'never' type when we already have 'void'. Though both types look similar, they represent two very different concepts.

A function that doesn't return a value implicitly returns the value undefined in JavaScript. Hence, even though we are saying it's not returning anything, it's returning 'undefined'. We usually ignore the return value in these cases. Such a function is inferred to have a void return type in TypeScript.

```
// This function returns undefined
function greet(name: string) {
console.log(`Hello, ${name}`);
}

let greeting = greet("David");
console.log(greeting);  // undefined
```

In contrast, a function that has a never return type never returns. It doesn't return undefined, either. There are 2 cases where functions should return never type:

1. In an unending loop e.g a while(true){} type loop.
2. A function that throws an error e.g function foo(){throw new Exception('Error message')}

14. Explain how enums work in TypeScript?

Enums allow us to create named constants. It is a simple way to give more friendly names to numeric constant values. An enum is defined by the keyword enum, followed by its name and the members.

Consider the following example that defines an enum Team with four values in it.

```
enum Team {
Alpha,
Beta,
Gamma,
Delta
}
let t: Team = Team.Delta;
```

By default, the enums start the numbering at 0. You can override the default numbering by explicitly assigning the values to its members.

TypeScript also lets you create enums with string values as follows:

```
enum Author {
  Anders = "Anders",
  Hejlsberg = "Hejlsberg"
};
```

15. What is the typeof operator? How is it used in TypeScript?

Similar to JavaScript, the typeof operator in TypeScript returns the type of the operand as a string.

```
console.log(typeof 10);  // "number"

console.log(typeof 'foo');  // "string"

console.log(typeof false);  // "boolean"

console.log(typeof bar);  // "undefined"
```

In TypeScript, you can use the typeof operator in a type context to refer to the type of a property or a variable.

```
let greeting = "hello";
let typeOfGreeting: typeof greeting;  // similar to let typeOfGreeting: string
```

16. What are the rest parameters and arguments in TypeScript?

A rest parameter allows a function to accept an indefinite number of arguments as an array. It is denoted by the '...' syntax and indicates that the function can accept one or more arguments.

```
function add(...values: number[]) {
let sum = 0;
values.forEach(val => sum += val);
return sum;
}
const sum = add(5, 10, 15, 20);
console.log(sum);  // 50
```


In contrast, the rest arguments allow a function caller to provide a variable

```
let typeOfGreeting: typeof greeting;  // similar to let typeOfGreeting: string
```

16. What are the rest parameters and arguments in TypeScript?

A rest parameter allows a function to accept an indefinite number of arguments as an array. It is denoted by the '...' syntax and indicates that the function can accept one or more arguments.

```
function add(...values: number[]) {
let sum = 0;
values.forEach(val => sum += val);
return sum;
}
const sum = add(5, 10, 15, 20);
console.log(sum);  // 50
```

In contrast, the rest arguments allow a function caller to provide a variable number of arguments from an array. Consider the following example.

```
const first = [1, 2, 3];
const second = [4, 5, 6];

first.push(...second);
console.log(first);  // [1, 2, 3, 4, 5, 6]
```

17. What is parameter destructuring?

Parameter destructing allows a function to unpack the object provided as an argument into one or more local variables.

```
function multiply({ a, b, c }: { a: number; b: number; c: number }) {
console.log(a * b * c);
}

multiply({ a: 1, b: 2, c: 3 });

You can simplify the above code by using an interface or a named type, as follows:
type ABC = { a: number; b: number; c: number };

function multiply({ a, b, c }: ABC) {
console.log(a * b * c);
}

multiply({ a: 1, b: 2, c: 3 });
```

18. Explain the TypeScript class syntax.

TypeScript fully supports classes. The TypeScript syntax for class declaration is similar to that of JavaScript, with the added type support for the member declarations.

Here is a simple class that defines an Employee type.

```
class Employee {
  name: string;
  salary: number;

  constructor(name: string, salary: number) {
          this.name = name;
          this.salary = salary;
  }
  promote() : void {
    this.salary += 10000;
  }
}
```

You can create an instance (or object) of a class by using the new keyword.

```
// Create a new employee
let john = new Employee("John", 60000);

console.log(john.salary);  // 60000
john.promote();
console.log(john.salary);  // 70000
```

19. Provide the syntax for optional parameters in TypeScript.

A function can mark one or more of its parameters as optional by suffixing its name with '?'. In the example below, the parameter greeting is marked optional.

```
function greet(name: string, greeting?: string) {
if (!greeting)
  greeting = "Hello";

console.log(`${greeting}, ${name}`);
}

greet("John", "Hi");  // Hi, John
greet("Mary", "Hola");  // Hola, Mary
greet("Jane");  // Hello, Jane
```

```
john.promote();
console.log(john.salary);  // 70000
```

19. Provide the syntax for optional parameters in TypeScript.

A function can mark one or more of its parameters as optional by suffixing its name with '?'. In the example below, the parameter greeting is marked optional.

```
function greet(name: string, greeting?: string) {
if (!greeting)
  greeting = "Hello";

console.log(`${greeting}, ${name}`);
}

greet("John", "Hi");  // Hi, John
greet("Mary", "Hola");  // Hola, Mary
greet("Jane");  // Hello, Jane
```

20. What is the purpose of the tsconfig.json file?

A tsconfig.json file in a directory marks that directory as the root of a TypeScript project. It provides the compiler options to compile the project.

Here is a sample tsconfig.json file:

```
{
 "compilerOptions": {
   "module": "system",
   "noImplicitAny": true,
   "removeComments": true,
   "outFile": "../../built/local/tsc.js",
   "sourceMap": true
 },
 "include": ["src/**/*"],
 "exclude": ["node_modules", "**/*.spec.ts"]
}
```

Typescript Interview Questions for Experienced

21. How to enforce strict null checks in TypeScript?

Null pointers are one of the most common sources of unexpected runtime errors in programming. TypeScript helps you avoid them to a large degree by enforcing strict null checks.

You can enforce strict null checks in two ways:

- providing the --strictNullChecks flag to the TypeScript (tsc) compiler
- setting the strictNullChecks property to true in the tsconfig.json configuration file.

When the flag is false, TypeScript ignores null and undefined values in the code. When it is true, null and undefined have their distinct types. The compiler throws a type error if you try to use them where a concrete value is expected.

22. Does TypeScript support static classes? If not, why?

TypeScript doesn't support static classes, unlike the popular object-oriented programming languages like C# and Java.

These languages need static classes because all code, i.e., data and functions, need to be inside a class and cannot exist independently. Static classes provide a way to allow these functions without associating them with any objects.

In TypeScript, you can create any data and functions as simple objects without creating a containing class. Hence TypeScript doesn't need static classes. A singleton class is just a simple object in TypeScript.

23. What are type assertions in TypeScript?

Sometimes, you as a programmer might know more about the type of a variable than TypeScript can infer. Usually, this happens when you know the type of an object is more specific than its current type. In such cases, you can tell the TypeScript compiler not to infer the type of the variable by using type assertions.

TypeScript provides two forms to assert the types.

- as syntax:

```
let value: unknown = "Foo";
let len: number = (value as string).length;
```

- <> syntax:

```
let value: unknown = "Foo";
let len: number = (<string>value).length;
```

Type assertions are similar to typecasting in other programming languages such as C# or Java. However, unlike those languages, there's no runtime penalty of boxing and unboxing variables to fit the types. Type assertions simply let the TypeScript compiler know the type of the variable.

24. Explain how tuple destructuring works in TypeScript.

You can destructure tuple elements by using the assignment operator (=).

the type of the variable.

24. Explain how tuple destructuring works in TypeScript.

You can destructure tuple elements by using the assignment operator (=). The destructuring variables get the types of the corresponding tuple elements.

```
let employeeRecord: [string, number] = ["John Doe", 50000];
let [emp_name, emp_salary] = employeeRecord;
console.log(`Name: ${emp_name}`);  // "Name: John Doe"
console.log(`Salary: ${emp_salary}`);  // "Salary: 50000"
```

After destructuring, you can't assign a value of a different type to the destructured variable. For example,

```
emp_name = true;  // Type 'boolean' is not assignable to type 'string'.(2322)
```

25. Explain the tuple types in TypeScript.

Tuples are a special type in TypeScript. They are similar to arrays with a fixed number of elements with a known type. However, the types need not be the same.

```
// Declare a tuple type and initialize it
let values: [string, number] = ["Foo", 15];

// Type 'boolean' is not assignable to type 'string'.(2322)
// Type 'string' is not assignable to type 'number'.(2322)
let wrongValues: [string, number] = [true, "hello"]; // Error
```

Since TypeScript 3.0, a tuple can specify one or more optional types using the ? as shown below.

```
let values: [string, number, boolean?] = ["Foo", 15];
```

26. What are type aliases? How do you create one?

Type aliases give a new, meaningful name for a type. They don't create new types but create new names that refer to that type.

For example, you can alias a union type to avoid typing all the types everywhere that value is being used.

```
type alphanumeric = string | number;
let value: alphanumeric = "";
value = 10;
```

27. What are intersection types?

Intersection types let you combine the members of two or more types by using the '&' operator. This allows you to combine existing types to get a single type with all the features you need.

The following example creates a new type Supervisor that has the members of types Employee and Manager.

```
interface Employee {
work: () => string;
}

interface Manager {
manage: () => string;
}

type Supervisor = Employee & Manager;

// john can both work and manage
let john: Supervisor;
```

28. What are union types in TypeScript?

A union type is a special construct in TypeScript that indicates that a value can be one of several types. A vertical bar (|) separates these types.

Consider the following example where the variable value belongs to a union type consisting of strings and numbers. The value is initialized to string "Foo". Because it can only be a string or a number, we can change it to a number later, and the TypeScript compiler doesn't complain.

```
let value: string | number = "Foo";
value = 10;  // Okay
```

However, if we try to set the value to a type not included in the union types, we get the following error.

```
value = true;  // Type 'boolean' is not assignable to type 'string | number'.(2322)
```

Union types allow you to create new types out of existing types. This removes a lot of boilerplate code as you don't have to create new classes and type hierarchies.

29. What are anonymous functions? Provide their syntax in TypeScript.

An anonymous function is a function without a name. Anonymous functions are typically used as callback functions, i.e., they are passed around to other functions, only to be invoked by the other function at a later point in time. For example,

```
setTimeout(function () {
  console.log('Run after 2 seconds')
}, 2000);
```

new classes and type hierarchies.

29. What are anonymous functions? Provide their syntax in TypeScript.

An anonymous function is a function without a name. Anonymous functions are typically used as callback functions, i.e., they are passed around to other functions, only to be invoked by the other function at a later point in time. For example,

```
setTimeout(function () {
  console.log('Run after 2 seconds')
}, 2000);

You can invoke an anonymous function as soon as it’s created. It’s called ‘immediately invoked fun
(function() {
  console.log('Invoked immediately after creation');
})();
```

30. What are abstract classes? When should you use one?

Abstract classes are similar to interfaces in that they specify a contract for the objects, and you cannot instantiate them directly. However, unlike interfaces, an abstract class may provide implementation details for one or more of its members.

An abstract class marks one or more of its members as abstract. Any classes that extend an abstract class have to provide an implementation for the abstract members of the superclass.

Here is an example of an abstract class Writer with two member functions. The write() method is marked as abstract, whereas the greet() method has an implementation. Both the FictionWriter and RomanceWriter classes that extend from Writer have to provide their specific implementation for the write method.

```
abstract class Writer {
abstract write(): void;

greet(): void {
  console.log("Hello, there. I am a writer.");
}
}

class FictionWriter extends Writer {
write(): void {
  console.log("Writing a fiction.");
}
}

class RomanceWriter extends Writer {
write(): void {
  console.log("Writing a romance novel.");
}
}

const john = new FictionWriter();
john.greet();  // "Hello, there. I am a writer."
john.write();  // "Writing a fiction."

const mary = new RomanceWriter();
mary.greet();  // "Hello, there. I am a writer."
mary.write();  // "Writing a romance novel."
```

31. How to make object properties immutable in TypeScript? (hint: readonly)

You can mark object properties as immutable by using the readonly keyword before the property name. For example:

```
interface Coordinate {
readonly x: number;
readonly y: number;
}
```

When you mark a property as readonly, it can only be set when you initialize the object. Once the object is created, you cannot change it.

```
let c: Coordinate = { x: 5, y: 15 };
c.x = 20; // Cannot assign to 'x' because it is a read-only property.(2540)
```

32. What is a type declaration file?

A typical TypeScript project references other third-party TypeScript libraries such as JQuery to perform routine tasks. Having type information for the library code helps you in coding by providing detailed information about the types, method signatures, etc., and provides IntelliSense.

A type declaration file is a text file ending with a .d.ts extension providing a way to declare the existence of some types or values without actually providing implementations for those values. It contains the type declarations but doesn’t have any source code. It doesn’t produce a .js file after compilation.

33. What are triple-slash directives?

Triple-slash directives are single-line comments that contain a single XML tag. TypeScript uses this XML tag as a compiler directive.

You can only place a triple-slash directive at the top of the containing file. C
triple-slash directive. TypeScript treats them as regular comments if it occu

doesn't produce a .js file after compilation.

33. What are triple-slash directives?

Triple-slash directives are single-line comments that contain a single XML tag. TypeScript uses this XML tag as a compiler directive.

You can only place a triple-slash directive at the top of the containing file. Only single or multi-line comments can come before a triple-slash directive. TypeScript treats them as regular comments if it occurs in the middle of a code block, after a statement.

The primary use of triple-slash directives is to include other files in the compilation process. For example, the following directive instructs the compiler to include a file specified by the path in the containing TypeScript file.

```
/// <reference path="..." />
```

Triple-slash directives also order the output when using --out or --outFile. The output files are produced to the output file location in the same order as the input files.

34. Explain the purpose of the 'in' operator.

The in operator is used to find if a property is in the specified object. It returns true if the property belongs to the object. Otherwise, it returns false.

```
const car = { make: 'Hyundai', model: 'Elantra', year: 2017 };
console.log('model' in car);  // true
console.log('test' in car);  // false
```

35. What are the 'implements' clauses in TypeScript?

An implements clause is used to check that a class satisfies the contract specified by an interface. If a class implements an interface and doesn't implement that interface, the TypeScript compiler issues an error.

```
interface Runnable {
run(): void;
}

class Job implements Runnable {
run() {
  console.log("running the scheduled job!");
}
}

// Class 'Task' incorrectly implements interface 'Runnable'.
// Property 'run' is missing in type 'Task' but required in type 'Runnable'.(2420)
class Task implements Runnable {
perform() {
  console.log("pong!");
}
}
```

A class can implement more than one interface. In this case, the class has to specify all the contracts of those interfaces.

36. What are string literal types?

In TypeScript, you can refer to specific strings and numbers as types.

```
let foo: "bar" = "bar";

// OK
foo = "bar";

// Error: Type '"baz"' is not assignable to type '"bar"'.(2322)
foo = "baz";
```

String literal types on their own are not that useful. However, you can combine them into unions. This allows you to specify all the string values that a variable can take, in turn acting like enums. This can be useful for function parameters.

```
function greet(name: string, greeting: "hi" | "hello" | "hola") {
// ...
}

greet("John", "hello");

// Error: Argument of type '"Howdy?"' is not assignable to parameter of type '"hi" | "hello" | "ho
greet("Mary", "Howdy?");
```

String literal types can help us spell-check the string values.

37. What are template literal types?

Template literal types are similar to the string literal types. You can combine them with concrete, literal types to produce a new string literal type. Template literal types allow us to use the string literal types as building blocks to create new string literal types.

```
type Point = "GraphPoint";

// type Shape = "Grid GraphPoint"
type Shape = `Grid ${Point}`;
```

Template literal types can also expand into multiple strings via unions. It he
each union member can represent.

String literal types can help us spell-check the string values.

37. What are template literal types?

Template literal types are similar to the string literal types. You can combine them with concrete, literal types to produce a new string literal type. Template literal types allow us to use the string literal types as building blocks to create new string literal types.

```
type Point = "GraphPoint";

// type Shape = "Grid GraphPoint"
type Shape = `Grid ${Point}`;
```

Template literal types can also expand into multiple strings via unions. It helps us create the set of every possible string literal that each union member can represent.

```
type Color = "green" | "yellow";
type Quantity = "five" | "six";

// type ItemTwo = "five item" | "six item" | "green item" | "yellow item"
type ItemOne = `${Quantity | Color} item`;
```

38. Explain the concept of inheritance in TypeScript.

Inheritance allows a class to extend another class and reuse and modify the behavior defined in the other class. The class which inherits another class is called the derived class, and the class getting inherited is called the base class.

In TypeScript, a class can only extend one class. TypeScript uses the keyword 'extends' to specify the relationship between the base class and the derived classes.

```
class Rectangle {
length: number;
breadth: number

constructor(length: number, breadth: number) {
  this.length = length;
  this.breadth = breadth
}

area(): number {
  return this.length * this.breadth;
}
}

class Square extends Rectangle {
constructor(side: number) {
  super(side, side);
}

volume() {
  return "Square doesn't have a volume!"
}
}

const sq = new Square(10);

console.log(sq.area());  // 100
console.log(sq.volume());  // "Square doesn't have a volume!"
```

In the above example, because the class Square extends functionality from Rectangle, we can create an instance of square and call both the area() and volume() methods.

39. What are conditional types? How do you create them?

A conditional type allows you to dynamically select one of two possible types based on a condition. The condition is expressed as a type relationship test.

```
C extends B ? TypeX : TypeY
```

Here, if type C extends B, the value of the above type is TypeX. Otherwise, it is TypeY.

40. What is the Function type in TypeScript?

Function is a global type in TypeScript. It has properties like bind, call, and apply, along with the other properties present on all function values.

```
function perform(fn: Function) {
fn(10);
}
```

You can always call a value of the Function type, and it returns a value of 'any' type.

41. List some of the utility types provided by TypeScript and explain their usage.

TypeScript provides various utility types that make common type transformations easy. These utility types are available globally. Here are some of the essential utility types included in TypeScript.

Utility Type	Description

You can always call a value of the Function type, and it returns a value of 'any' type.

41. List some of the utility types provided by TypeScript and explain their usage.

TypeScript provides various utility types that make common type transformations easy. These utility types are available globally. Here are some of the essential utility types included in TypeScript.

Utility Type	Description
Partial<Type>	Constructs a type with all properties of Type set to optional.
Required<Type>	Constructs a type consisting of all properties of Type set to required.
Readonly<Type>	Constructs a type with all properties of Type set to readonly.
Record<Keys, Type>	Constructs an object type with property keys are of type Keys, and values are Type.

42. Explain the various ways to control member visibility in TypeScript.

TypeScript provides three keywords to control the visibility of class members, such as properties or methods.

- **public:** You can access a public member anywhere outside the class. All class members are public by default.
- **protected:** A protected member is visible only to the subclasses of the class containing that member. Outside code that doesn't extend the container class can't access a protected member.
- **private:** A private member is only visible inside the class. No outside code can access the private members of a class.

43. Explain the different variants of the for loop in TypeScript.

TypeScript provides the following three ways to loop over collections.

- 'for' loop

```
let values = [10, "foo", true];

for(let i=0; i<values.length; i++) {
  console.log(values[i]);  // 10, "foo", true
}
```

- 'forEach' function

```
let values = [10, "foo", true];
values.forEach(val => {
    console.log(val);  // 10, "foo", true
})
```

- 'for..of' statement

```
let values = [10, "foo", true];
for (let val of values) {
console.log(val); // 10, "foo", true
}
```

44. Explain the symbol type in TypeScript.

Symbols were introduced in ES6 and are supported by TypeScript. Similar to numbers and strings, symbols are primitive types. You can use Symbols to create unique properties for objects.

You can create symbol values by calling the Symbol() constructor, optionally providing a string key.

```
let foo = Symbol();
let bar = Symbol("bar"); // optional string key
```

A key characteristic of symbols is that they are unique and immutable.

```
let foo = Symbol("foo");
let newFoo = Symbol("foo");

let areEqual = foo === newFoo;
console.log(areEqual);  // false, symbols are unique
```

45. Explain how optional chaining works in TypeScript.

Optional chaining allows you to access properties and call methods on them in a chain-like fashion. You can do this using the '?.' operator.

TypeScript immediately stops running some expression if it runs into a 'null' or 'undefined' value and returns 'undefined' for the entire expression chain.

Using optional chaining, the following expression

```
let x = foo === null || foo === undefined ? undefined : foo.bar.baz();
```

can be expressed as:

```
let x = foo?.bar.baz();
```

46. Provide the TypeScript syntax to create function overloads.

Function overloading allows us to define multiple functions with the same n
of parameters are different.

```
let x = foo?.bar.baz();
```

46. Provide the TypeScript syntax to create function overloads.

Function overloading allows us to define multiple functions with the same name, as long as their number of parameters or the types of parameters are different.

The following example defines two overloads for the function buildDate. The first overload takes a number as a parameter, whereas the second takes three numbers as parameters. These are called overload signatures.

The body of the function also called an implementation signature, follows the overload signatures. You can't call this signature directly, as it's not visible from the outside. It should be compatible with the overload signatures.

```
function buildDate(timestamp: number): Date;
function buildDate(m: number, d: number, y: number): Date;
function buildDate(mOrTimestamp: number, d?: number, y?: number): Date {
if (d !== undefined && y !== undefined) {
  return new Date(y, mOrTimestamp, d);
} else {
  return new Date(mOrTimestamp);
}
}

const d1 = buildDate(87654321);
const d2 = buildDate(2, 2, 2);
```

47. What is meant by type inference?

TypeScript can infer the type of a variable when you don't provide an explicit type. This is known as type inference. This is usually done when the variables or parameters are initialized during the declaration.

For example, TypeScript knows that the variable foo is a string, even though we don't mention string as a type.

```
let foo = "this is a string";
console.log(typeof foo);  // "string"
```

48. What is meant by contextual typing?

When the TypeScript compiler uses the location (or context) of a variable to infer its type, it's called contextual typing.

In the following example, TypeScript uses the Window.onmousedown function type information to infer the type of the function expression on the right-hand side of the assignment. This allows it to infer the type of the e parameter, which does have a button property but not a property named foo.

```
window.onmousedown = function (e) {
console.log(e.button); //<- OK
console.log(e.foo); //<- Error!
};
```

49. What is the purpose of noImplicitAny?

Usually, when we don't provide any type on a variable, TypeScript assumes 'any' type. For example, TypeScript compiles the following code, assuming the parameter '**s**' is of any type. It works as long as the caller passes a string.

```
function parse(s) {
console.log(s.split(' '));
}
parse("Hello world");  // ["Hello", "world"]
```

However, the code breaks down as soon as we pass a number or other type than a string that doesn't have a split() method on it. For example,

```
function parse(s) {
console.log(s.split(' '));  // [ERR]: s.split is not a function
}
parse(10);
```

noImplicitAny is a compiler option that you set in the tsconfig.json file. It forces the TypeScript compiler to raise an error whenever it infers a variable is of any type. This prevents us from accidentally causing similar errors.

```
Parameter 's' implicitly has an 'any' type.(7006)
function parse(s) {
console.log(s.split(' '));  // [ERR]: s.split is not a function
}
```

50. What is an interface?

An interface defines a contract by specifying the type of data an object can have and its operations. In TypeScript, you can specify an object's shape by creating an interface and using it as its type. It's also called "duck typing".

In TypeScript, you can create and use an interface as follows:

```
interface Employee {
name: string;
salary: number;
}

function process(employee: Employee) {
console.log(`${employee.name}'s salary = ${employee.sal
```

Usually, when we don't provide any type on a variable, TypeScript assumes 'any' type. For example, TypeScript compiles the following code, assuming the parameter **'s'** is of any type. It works as long as the caller passes a string.

```
function parse(s) {
console.log(s.split(' '));
}
parse("Hello world");  // ["Hello", "world"]
```

However, the code breaks down as soon as we pass a number or other type than a string that doesn't have a split() method on it. For example,

```
function parse(s) {
console.log(s.split(' '));  // [ERR]: s.split is not a function
}
parse(10);
```

noImplicitAny is a compiler option that you set in the tsconfig.json file. It forces the TypeScript compiler to raise an error whenever it infers a variable is of any type. This prevents us from accidentally causing similar errors.

```
Parameter 's' implicitly has an 'any' type.(7006)
function parse(s) {
console.log(s.split(' '));  // [ERR]: s.split is not a function
}
```

50. What is an interface?

An interface defines a contract by specifying the type of data an object can have and its operations. In TypeScript, you can specify an object's shape by creating an interface and using it as its type. It's also called "duck typing".

In TypeScript, you can create and use an interface as follows:

```
interface Employee {
name: string;
salary: number;
}

function process(employee: Employee) {
console.log(`${employee.name}'s salary = ${employee.salary}`);
}

let john: Employee = {
name: "John Doe",
salary: 150000
}

process(john);  // "John Doe's salary = 150000"
```

Interfaces are an effective way to specify contracts within your code as well as outside your code.

Conclusion

51. Conclusion

TypeScript has been increasing in popularity for the last few years, and many large organizations and popular frameworks have adopted TypeScript to manage their large JavaScript codebases. It's a valuable skill to have as a developer.

In this article, we explored the questions that a developer might get asked in an interview. We have provided simple code examples to cement the concepts further. Finally, you can use the multiple-choice questions at the end of the article to test your understanding of the various topics in TypeScript.

Reference:

- TypeScript Documentation
- Angular Interview Questions and Answers
- JavaScript Interview Questions
- Difference between Typescript and Javascript

TypeScript MCQ Questions

1.The TypeScript was created by

- ◯ Guido van Rossum
- ◯ Rasmus Lerdorf
- ◯ Anders Hejlsberg
- ◯ Ken Thompson

2.Which of the following files controls the TypeScript configuration?

- ◯ jsconfig.json
- ◯ typescript.config